ICPC Team Reference

CoogLimitExceeded

Contents

Bitset Funcs **2**

Dijkstra **3**

Egcd **4**

Floyd Warshall **5**

Getline Input **7**

Getline Splitting **8**

Modinv Fastpower And Mod Comb **9**

Multiset Funcs **10**

Ordered Set Lib **11**

Python Fast Input **12**

Segment Tree **13**

Sieve Primes **15**

Smallest Prime Factor **16**

String Hash **17**

Template Code **19**

Trie **20**

Vector Funcs **21**

Bitset Funcs

```cpp
// bit_utils.hpp
#pragma once
#include <bits/stdc++.h>
using namespace std;
#define ll long long

// Convert string of bits ("1011") to long long
ll bitset_to_ll(const string &s) {
    return (ll)bitset<64>(s).to_ullong();
}

// Convert long long to bitset string of length n
string ll_to_bitset(ll x, int n = 64) {
    bitset<64> b(x);
    string s = b.to_string();
    return s.substr(64 - n, n);
}

// Count number of set bits
int count_bits(ll x) {
    return (int)bitset<64>(x).count();
}

// Check if k-th bit (0-indexed from right) is set
bool get_bit(ll x, int k) {
    return (x >> k) & 1LL;
}

// Set k-th bit
ll set_bit(ll x, int k) {
    return x | (1LL << k);
}

// Clear k-th bit
ll clear_bit(ll x, int k) {
    return x & ~(1LL << k);
}

// Toggle k-th bit
ll toggle_bit(ll x, int k) {
    return x ^ (1LL << k);
}

// Get lowest set bit position (1-indexed). Returns 0 if none.
int lowest_set_bit_pos(ll x) {
    if (x == 0) return 0;
    return __builtin_ctzll(x) + 1;
}

// Get highest set bit position (1-indexed). Returns 0 if none.
int highest_set_bit_pos(ll x) {
    if (x == 0) return 0;
    return 64 - __builtin_clzll(x);
}
```

Dijkstra

```cpp
#include <bits/stdc++.h>
using namespace std;
#define ll long long

struct edge {
    int node; ll weight; //node on the end of edge, weight of edge
};
// to customize the priority queue, we can use a custom comparator
bool operator < (edge a, edge b){
    return a.weight > b.weight; // O(1)
}
const int N = 1e5;
ll distance_to_node[N];
int parent[N];
vector<edge> graph[N];

void dijkstra(int start, int destination)
{
    memset(distance_to_node,63,sizeof distance_to_node); // O(N)
    priority_queue<edge>pq; // O(1)
    pq.push({start,0}); // O(log 1) = O(1)
    distance_to_node[start] = 0; parent[start] = -1; // O(1)

    while(!pq.empty()){ // loop runs up to O((V + E) log V) overall
        auto current = pq.top(); pq.pop(); // O(log V)

        if (current.node == destination) break; // O(1)

        for (auto e : graph[current.node]){ // iterates over neighbors
    -> O(E) total across all loops
            ll new_weight = current.weight + e.weight; // O(1)
            if (distance_to_node[e.node] > new_weight) { // O(1)
                distance_to_node[e.node] = new_weight; // O(1)
                pq.push({e.node, new_weight}); // O(log V)
                parent[e.node] = current.node; // O(1)
            }
        }
    }
  //no path
    if (parent[destination] == 0)
      return; // O(1)

    vector<int> path; // O(1)
    int cur = destination; // O(1)
    while(cur != -1){ // O(V) in worst case
        path.push_back(cur); // O(1) amortized
        cur = parent[cur]; // O(1)
    }
    reverse(path.begin(), path.end()); // O(V)
}
```

Egcd

```cpp
#include <bits/stdc++.h>
using namespace std;

#define ll long long

// Extended Euclidean Algorithm
// Returns gcd(a, b) and finds x, y such that a*x + b*y = gcd(a, b)
ll egcd(ll a, ll b, ll &x, ll &y) {
    if (b == 0) {          // Base case
        x = 1; y = 0;
        return a;
    }
    ll x1, y1;
    ll g = egcd(b, a % b, x1, y1);  // Recurse
    x = y1;                         // Update x
    y = x1 - (a / b) * y1;          // Update y
    return g;                       // Return gcd
}
// Time complexity: O(log(max(a, b)))
```

Floyd Warshall

```cpp
#include <bits/stdc++.h>
using namespace std;

// --- Your "edge" struct (assuming you have this) ---
struct edge {
    int from, to, distance;
};

// --- ICPC-Style Floyd-Warshall Library Code ---

// !! Change this based on problem constraints !!
const int MAXN = 103; // Max number of nodes + 1 (for 1-indexing)
using ll = long long;
const ll INF = 1e18;

struct FloydWarshall {
    ll dp[MAXN][MAXN];
    int next_hop[MAXN][MAXN];
    int N; // Actual number of nodes

    // Initialize for 'n' nodes (1-indexed)
    FloydWarshall(int nodes) {
        N = nodes;
        for (int i = 1; i <= N; i++) {
            for (int j = 1; j <= N; j++) {
                dp[i][j] = (i == j) ? 0 : INF;
                next_hop[i][j] = -1; // -1 means no path
            }
            next_hop[i][i] = i;
        }
    }

    // Add a directed edge
    void add_edge(int u, int v, ll w) {
        if (w < dp[u][v]) { // Handle parallel edges
            dp[u][v] = w;
            next_hop[u][v] = v; // The first hop from u to v is v
        }
    }

    // Run the O(N^3) algorithm
    void run() {
        for (int k = 1; k <= N; k++) {
            for (int i = 1; i <= N; i++) {
                for (int j = 1; j <= N; j++) {
                    // Check for INF to prevent overflow
                    if (dp[i][k] == INF || dp[k][j] == INF) continue;

                    ll new_dist = dp[i][k] + dp[k][j];
                    if (new_dist < dp[i][j]) {
                        dp[i][j] = new_dist;
                        // The first hop from i->j is the same as the first hop from i->k
                        next_hop[i][j] = next_hop[i][k];
                    }
                }
```

```
            }
        }
    }

    // Get the shortest path distance from u to v
    ll get_dist(int u, int v) {
        return dp[u][v]; // Will be INF if no path
    }

    // Reconstruct the path from u to v
    vector<int> get_path(int u, int v) {
        if (dp[u][v] == INF) {
            return {}; // No path
        }

        vector<int> path;
        int curr = u;
        while (curr != v) {
            if (curr == -1) return {}; // Should not happen if dp[u][v] != INF
            path.push_back(curr);
            curr = next_hop[curr][v];
        }
        path.push_back(v);
        return path;
    }

    // Check for negative cycles
    bool has_negative_cycle() {
        for (int i = 1; i <= N; i++) {
            if (dp[i][i] < 0) {
                return true;
            }
        }
        return false;
    }
};
```

Getline Input

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    // --- Example 1: Multiple lines in a loop ---
    int n = 3; // number of lines
    cout << "Enter " << n << " lines:\n";
    for (int i = 0; i < n; i++) {
        string line;
        getline(cin, line);
        cout << "Line " << i+1 << ": " << line << "\n";
    }

    // --- Example 2: Number first, then lines ---
    int m;
    cout << "Enter number of lines: ";
    cin >> m;
    cin.ignore(numeric_limits<streamsize>::max(), '\n'); // discard leftover '\n'

    cout << "Enter " << m << " lines:\n";
    for (int i = 0; i < m; i++) {
        string line;
        getline(cin, line);
        cout << "Line " << i+1 << ": " << line << "\n";
    }

    // --- Example 3: Reading until EOF using while(getline) ---
    cout << "Enter lines (Ctrl+D / Ctrl+Z to stop):\n";
    string line;
    while (getline(cin, line)) {
        cout << "Read: " << line << "\n";
    }

    return 0;
}
```

Getline Splitting

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    string s = "apple,banana,orange,grape";
    vector<string> tokens;
    string token;

    stringstream ss(s);
    while (getline(ss, token, ',')) { // split by comma
        tokens.push_back(token);
    }

    for (const auto &t : tokens)
        cout << t << "\n";

    return 0;
}
```

Modinv Fastpower And Mod Comb

```cpp
#include <bits/stdc++.h>
using namespace std;

#define ll long long

ll mod = 1e9 + 7;
ll fact[2009], inv[2009];

// Fast exponentiation: b^e mod m
ll fastPower(ll b, ll e, ll m) {
    if (e == 0) return 1;
    ll hp = fastPower(b, e / 2, m);
    hp = (hp * hp) % m;
    if (e % 2) hp = (hp * b) % m;
    return hp % m;
}

// Modular inverse using Fermat's little theorem
ll modularInverse(ll b, ll m) {
    return fastPower(b, m - 2, m);
}

// Precompute factorials and inverses up to mx
void factorials(int mx) {
    fact[0] = inv[0] = 1;
    for (int i = 1; i < mx; i++) {
        fact[i] = (i * fact[i - 1]) % mod;
        inv[i] = modularInverse(fact[i], mod);
    }
}

// Compute nCr mod m
ll nCr(ll n, ll r, ll m) {
    if (r < 0 || r > n) return 0;
    ll a = fact[n];
    ll b = (inv[n - r] * inv[r]) % m;
    return (a * b) % m;
}
```

Multiset Funcs

```cpp
#include <bits/stdc++.h>
using namespace std;

// Default (ascending) multiset behavior
void multiset_default() {
    vector<int> values = {15, 3, 22, 11, 5, 18, 7};

    multiset<int> ms(values.begin(), values.end());
    // ms = {3, 5, 7, 11, 15, 18, 22}

    ms.insert(11); // duplicates allowed
    // ms = {3, 5, 7, 11, 11, 15, 18, 22}

    ms.count(11); // 2
    ms.find(11);  // points to first 11
    ms.erase(11); // removes all 11s by default
    // ms = {3, 5, 7, 15, 18, 22}

    auto lb = ms.lower_bound(11); // first element >= 11 -> 15
    auto ub = ms.upper_bound(11); // first element > 11 -> 15

    // Iteration is in ascending order
    // elements printed: 3 5 7 15 18 22
}

// Descending comparator (equivalent to greater<int>)
struct Descending {
    bool operator()(int a, int b) const {
        return a > b;
    }
};

void multiset_descending() {
    vector<int> values = {15, 3, 22, 11, 5, 18, 7};

    // equivalent: multiset<int, greater<int>> ms_desc;
    multiset<int, Descending> ms_desc(values.begin(), values.end());
    // ms_desc = {22, 18, 15, 11, 7, 5, 3}

    ms_desc.insert(11); // duplicates allowed
    // ms_desc = {22, 18, 15, 11, 11, 7, 5, 3}

    int target = 11;
    auto lb = ms_desc.lower_bound(target);
    // lower_bound(11) = first element 'x' such that !(x > 11) (i.e., x <= 11) -> 11 (first one)
    auto ub = ms_desc.upper_bound(target);
    // upper_bound(11) = first element 'x' such that !(x >= 11) (i.e., x < 11) (in descending order) -> 7

    // Iteration is in descending order
    // elements printed: 22 18 15 11 11 7 5 3
}
```

Ordered Set Lib

```cpp
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using namespace std;
using namespace __gnu_pbds;

// Ordered set template
template<typename T>
using ordered_set = tree<T, null_type, less<T>, rb_tree_tag,
    tree_order_statistics_node_update>;

int main() {
    // Initialize (O(n log n) if multiple inserts)
    ordered_set<int> os;
    os.insert(5);
    os.insert(1);
    os.insert(3);
    os.insert(7);

    // Find k-th smallest element (0-indexed) - O(log n)
    cout << *os.find_by_order(0) << " "  // 1
         << *os.find_by_order(2) << "\n"; // 5

    // Number of elements < x - O(log n)
    cout << os.order_of_key(5) << " "   // 2
         << os.order_of_key(6) << "\n"; // 3

    // Erase element - O(log n)
    os.erase(3);

    // Iterate elements - O(n)
    for (int x : os) cout << x << " ";  // 1 5 7
    cout << "\n";

    // Insert element - O(log n)
    os.insert(4);

    // Check order after insert
    cout << *os.find_by_order(2) << "\n"; // 4

    return 0;
}
```

Python Fast Input

```python
# FAST standard input
import sys
input = sys.stdin.readline

n = int(input())
arr = list(map(int, input().split()))
sys.stdout.write(str(sum(arr)) + "\n")

# FOR FILES
with open("input.txt", "r") as fin, open("output.txt", "w") as fout:
    n = int(fin.readline())
    arr = list(map(int, fin.readline().split()))
    fout.write(str(sum(arr)) + "\n")
```

Segment Tree

```
// templated node type.
template <typename T>
struct node
{
    T val;
};

// generic segment tree storing nodes of type node<T>
template <typename T>
struct segment_tree
{
    int size = 1;
    vector<node<T>> nodes;

    // for safe margin and no loop. we can initialize by 4*num_leaves
    // NOT RECOMMENDED for multiple initialization or large N
    void init(int num_leaves)
    {
        while (size < num_leaves)
            size <<= 1;
        nodes.resize(size << 1);
        nodes.assign(size << 1, neutral_node());
    }

    node<T> neutral_node() const
    {
        return {T{}}; // like 0 or empty string
    }

    node<T> single(T val) const
    {
        return {val};
    }

    // merge operation - here sum for generic T that supports +, it
    can be product, min, max, etc.
    node<T> merge(const node<T> &node1, const node<T> &node2) const
    {
        return single(node1.val + node2.val);
    }

    void build(vector<T> &v, int x, int lx, int rx)
    {
        if (rx - lx == 1)
        {
            if (lx < (int)v.size())
            {
                nodes[x] = single(v[lx]);
            }
            return;
        }

        int m = (rx + lx) >> 1;
        build(v, 2 * x + 1, lx, m);
        build(v, 2 * x + 2, m, rx);
        nodes[x] = merge(nodes[2 * x + 1], nodes[2 * x + 2]);
```

```
    }

    void build(vector<T> &v)
    {
        build(v, 0, 0, size);
    }

    // desired index, value (as node), current node index, left and right bounds of node
    void set_index_to_value(int i, const node<T> &v, int x, int lx, int rx)
    {
        if (rx - lx == 1)
        { // we are at desired node
            nodes[x] = v;
            return;
        }

        int m = (rx + lx) >> 1;
        if (i < m)
        { // go left
            set_index_to_value(i, v, 2 * x + 1, lx, m);
        }
        else
        { // go right
            set_index_to_value(i, v, 2 * x + 2, m, rx);
        }
        nodes[x] = merge(nodes[2 * x + 1], nodes[2 * x + 2]);
    }

    void set_index_to_value(int i, T val)
    {
        set_index_to_value(i, single(val), 0, 0, size);
    }

    node<T> get(int l, int r, int x, int lx, int rx) const
    {
        if (l >= rx || lx >= r)
        { // out of bounds of query
            return neutral_node();
        }
        if (lx >= l && rx <= r)
        {
            return nodes[x]; // all within bounds, take this node
        }

        int m = (rx + lx) >> 1;
        node<T> lt = get(l, r, 2 * x + 1, lx, m);
        node<T> rt = get(l, r, 2 * x + 2, m, rx);
        return merge(lt, rt);
    }

    node<T> get(int l, int r) const
    {
        return get(l, r, 0, 0, size);
    }
};
```

Sieve Primes

```cpp
#include <bits/stdc++.h>
using namespace std;

const int N = 1e5;
bool prime[N];

// Sieve of Eratosthenes: marks prime[i] = true if i is prime
// Time: O(N log log N), Space: O(N)
void sieve(int X) {
    memset(prime, true, sizeof prime);
    prime[0] = prime[1] = false;
    for (int i = 2; i * i <= X; i++) {
        if (prime[i]) {
            for (int j = i * i; j <= X; j += i) prime[j] = false;
        }
    }
}
```

Smallest Prime Factor

```cpp
#include <bits/stdc++.h>
using namespace std;

const int N = 1e7;        // Maximum number for preprocessing

int spf[N] = {};           // Smallest prime factor array

// Preprocess smallest prime factors for all numbers up to N
void preprocess() {
    spf[0] = spf[1] = 0;          // 0 and 1 are not prime
    for (int i = 2; i < N; ++i) spf[i] = i;  // Initialize spf[i] = i

    for (int i = 2; i * i < N; ++i) {
        if (spf[i] == i) {            // i is prime
            for (int j = i * i; j < N; j += i)
                if (spf[j] > i) spf[j] = i;  // Update smallest prime factor
        }
    }
}

// Time complexity: O(N log log N) using sieve
// Space complexity: O(N)

/* USAGE:

int x = 60;
vector<int> factors;
while (x > 1) {
    factors.push_back(spf[x]); // get smallest prime factor
    x /= spf[x];               // reduce x
}
*/
```

String Hash

```cpp
/*
USAGE:
    - Adjust M (mod) or base primes (P1, P2) as needed.
    - Call pre() once before all tests to precompute powers and inverses.
    - Construct Hash(s) for a string.
    - Use getRangeHashVal(l, r) for inclusive substring hash.
    Hashing supports double mod-bases (P1, P2) to reduce collisions.
*/

#include <bits/stdc++.h>
using namespace std;

// Constants
const int N = 3 * 1e5 + 5, P1 = 31, P2 = 37, M = 1e9 + 7;
int pw1[N], pw2[N], inv1[N], inv2[N];

// Modular multiplication and addition
void apply_mod(int &a)
{
    a = ((a % M) + M) % M;
}
int mul(int a, int b)
{
    apply_mod(a);
    apply_mod(b);
    return (1LL * a * b) % M;
}
int add(int a, int b)
{
    apply_mod(a);
    apply_mod(b);
    return (a + b) % M;
}

// Rolling hash with prefix storage
struct Hash
{
    vector<pair<int, int>> prefixHash;

    // O(n): Precompute prefix hash values
    Hash(string s)
    {
        prefixHash = vector<pair<int, int>>(s.size(), {0, 0});
        for (int i = 0; i < (int)s.size(); i++)
        {
            prefixHash[i].first = mul(s[i] - 'a' + 1, pw1[i]);
            prefixHash[i].second = mul(s[i] - 'a' + 1, pw2[i]);
            if (i)
                prefixHash[i] = {
                    add(prefixHash[i].first, prefixHash[i - 1].first),
                    add(prefixHash[i].second, prefixHash[i - 1].second
)};
        }
    }

```

```
55      // Return full string hash
56      pair<int, int> getHashVal()
57      {
58          return prefixHash.back();
59      }
60
61      // O(1): Substring hash for inclusive range [l, r]
62      pair<int, int> getRangeHashVal(int l, int r)
63      {
64          return {
65              mul(add(prefixHash[r].first, -(l ? prefixHash[l - 1].first
        : 0)), inv1[l]),
66              mul(add(prefixHash[r].second, -(l ? prefixHash[l - 1].
       second : 0)), inv2[l])};
67      }
68  };
69
70  // Fast modular exponentiation - O(log power)
71  int fastPower(int base, int power)
72  {
73      if (!power)
74          return 1;
75      int ret = fastPower(base, power >> 1);
76      ret = mul(ret, ret);
77      if (power & 1)
78          ret = mul(ret, base);
79      return ret;
80  }
81
82  // O(N): Precompute powers and modular inverses
83  void preprocess()
84  {
85      pw1[0] = inv1[0] = pw2[0] = inv2[0] = 1;
86      int mulInv1 = fastPower(P1, M - 2);
87      int mulInv2 = fastPower(P2, M - 2);
88      for (int i = 1; i < N; i++)
89      {
90          pw1[i] = mul(pw1[i - 1], P1);
91          pw2[i] = mul(pw2[i - 1], P2);
92          inv1[i] = mul(inv1[i - 1], mulInv1);
93          inv2[i] = mul(inv2[i - 1], mulInv2);
94      }
95  }
```

Template Code

```cpp
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using namespace std;
using namespace __gnu_pbds;
template<typename T> using ordered_set = tree<T, null_type, less<T>,
    rb_tree_tag, tree_order_statistics_node_update>;
typedef long long ll;
typedef vector<int> vi;
typedef vector<ll> vll;

#define el '\n'
#define pb push_back
#define all(x) (x).begin(),(x).end()
#define rall(x) (x).rbegin(),(x).rend()
#define OO (int)2e9
#define LOO (ll)9e18
#define bitCountLL(x) __builtin_popcountll(x)
#define bitCount(x) __builtin_popcount(x)

int di[] = {-1, 1, 0, 0, 1, -1, 1, -1};
int dj[] = {0, 0, 1, -1, 1, -1, -1, 1};


const int mod = (int) 1e9 + 7;
const int N = 2e5 + 5, M = 20 + 5;

bool MULTIPLE_TEST_CASES = true;

void run_case(int case_index){
    cout << "Hello world" << endl;
}

int main() {
    // FILE IO
    freopen("input.txt", "r", stdin);
    freopen("output.txt", "w", stdout);

    // FAST IO
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);cout.tie(nullptr);

    int test_cases = 1;
    if (MULTIPLE_TEST_CASES) cin >> test_cases;

    for (int i = 1; i <= test_cases; ++i) {
        run_case(i);
    }
}
```

Trie

```
#include <bits/stdc++.h>
using namespace std;


//TRIE

struct Node {
    Node *next_node[26] = {};
    int freq[26] = {}; //to count frequencies of some prefix
    Node() {
        memset(next_node, 0, sizeof next_node);
        memset(freq, 0, sizeof freq);
    }
};

Node *root;

void insert(string s) {
    Node *ptr = root;
    for (int i = 0; i < (int)s.size(); i++) {
        int c = s[i] - 'a';
        if (!ptr->next_node[c])
            ptr->next_node[c] = new Node();
        ptr->freq[c]++;
        ptr = ptr->next_node[c];
    }
}

//count frequency of given prefix
int count(string s) {
    Node *ptr = root;
    for (int i = 0; i < (int)s.size(); i++) {
        int c = s[i] - 'a';
        if (!ptr->next_node[c])
            return 0;
        if (i == (int)s.size() - 1)
            return ptr->freq[c];
        ptr = ptr->next_node[c];
    }
    return 0;
}

/*
 USAGE
 root = new Node();
// insert strings, count them if 0 it DNE

  */
```

Vector Funcs

```cpp
#include <bits/stdc++.h>
using namespace std;

#define all(x) x.begin(), x.end()

int main() {
    vector<int> base = {5, 1, 3, 9, 7, 3};

    // accumulate (sum) and partial_sum (prefix sums)
    {
        vector<int> v = base;
        accumulate(all(v), 0); // sum = 28
        vector<int> pref(v.size());
        partial_sum(all(v), pref.begin()); // pref = {5, 6, 9, 18, 25, 28}
    }

    // count, find, any_of / all_of / none_of (O(n) time)
    {
        vector<int> v = base;
        count(all(v), 3);             // 2
        any_of(all(v), [](int x){return x > 8;}); // true
        all_of(all(v), [](int x){return x < 10;}); // true
        none_of(all(v), [](int x){return x < 0;}); // true
    }

    // lower_bound / upper_bound (O(log n) on sorted vector)
    {
        vector<int> v = base;
        sort(all(v));                 // v = {1, 3, 3, 5, 7, 9}
        int x = 3;
        auto lb = lower_bound(all(v), x); // index = 1
        auto ub = upper_bound(all(v), x); // index = 3
    }

    // "Inverse" (descending) logic
    // lower_bound / upper_bound (O(log n) on sorted vector)
    {
        vector<int> v = base;
        sort(all(v), greater<int>());   // v = {9, 7, 5, 3, 3, 1}
        int x = 3;

        // MUST pass greater<int>() to lower_bound
        auto lb = lower_bound(all(v), x, greater<int>());
        // Logic: first 'x' such that !(x > 3), i.e., x <= 3
        // To get index: distance(v.begin(), lb) -> 3 (points to the first 3)

        // MUST pass greater<int>() to upper_bound
        auto ub = upper_bound(all(v), x, greater<int>());
        // Logic: first 'x' such that (3 > x), i.e., x < 3
        // To get index: distance(v.begin(), ub) -> 5 (points to the 1)
    }

    // unique + erase (deduplicate sorted vector, O(n) time)
```

```cpp
    {
        vector<int> v = base;
        sort(all(v));                 // v = {1, 3, 3, 5, 7, 9}
        v.erase(unique(all(v)), v.end()); // v = {1, 3, 5, 7, 9}
    }

    // next_permutation / prev_permutation (O(n) time)
    {
        vector<int> v = {1, 2, 3};
        next_permutation(all(v));     // v = {1, 3, 2}
        prev_permutation(all(v));     // v = {1, 2, 3}
    }

    // nth_element (k-th order statistic, avg O(n))
    {
        vector<int> v = base;
        int k = 2;
        nth_element(v.begin(), v.begin() + k, v.end());
        v[k]; // 3 (3rd smallest element)
    }

    // sort with custom comparator (O(n log n))
    {
        vector<int> v = base;
        sort(all(v), [](int a, int b){
            if(a % 10 == b % 10) return a < b;
            return (a % 10) < (b % 10);
        });
        // v = {1, 3, 3, 5, 7, 9} (sorted by last digit, ties by value)
    }

    // set operations on sorted vectors (O(n + m) time)
    {
        vector<int> a = {1, 2, 3, 4, 5};
        vector<int> b = {3, 4, 5, 6, 7};
        vector<int> res;

        set_union(a.begin(), a.end(), b.begin(), b.end(), back_inserter(res));
        // res = {1, 2, 3, 4, 5, 6, 7}

        res.clear();
        set_intersection(a.begin(), a.end(), b.begin(), b.end(), back_inserter(res));
        // res = {3, 4, 5}

        res.clear();
        set_difference(a.begin(), a.end(), b.begin(), b.end(), back_inserter(res));
        // res = {1, 2}
    }

    return 0;
}
```